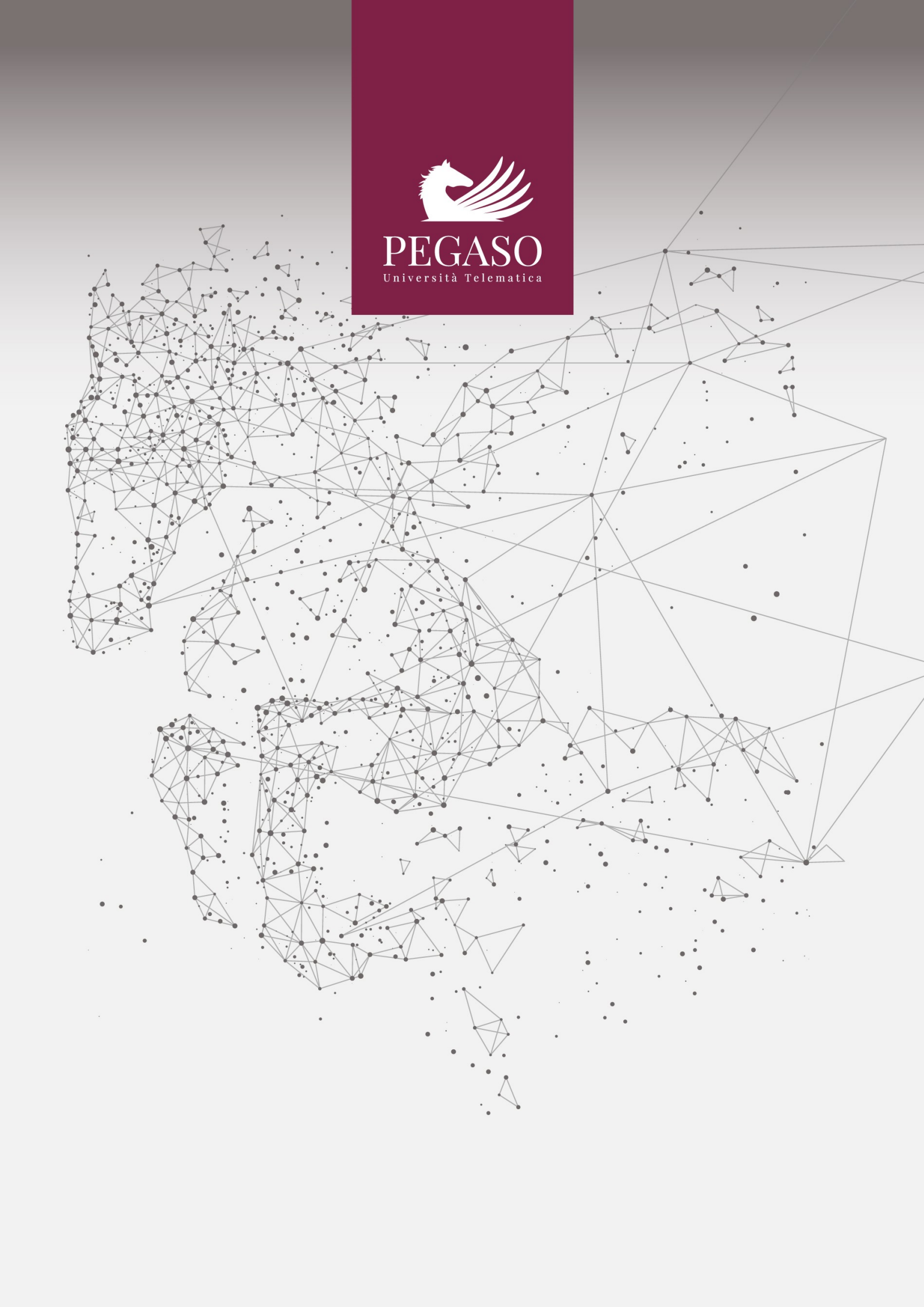

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. ALGORITMO CENTRALIZZATO DI MUTUA ESCLUSIONE 4
3. ALGORITMO DISTRIBUITO DI MUTUA ESCLUSIONE....6
4. ALGORITMO TOKEN-RING....9
BIBLIOGRAFIA 11

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

La concorrenza e la collaborazione di più processi sono fondamentali per i sistemi distribuiti. In molti casi, ciò richiede anche l'accesso simultaneo alle stesse risorse da parte di più processi. Per evitare che gli accessi simultanei danneggino o rendano inconsistente la risorsa, è necessario concedere l'accesso mutuamente esclusivo ai processi. In questa lezione, esaminiamo alcuni degli algoritmi distribuiti più significativi e rappresentativi proposti.

Gli algoritmi per la mutua esclusione distribuita possono essere suddivisi in due categorie distinte. Nelle soluzioni basate su token, la mutua esclusione si ottiene passando un token, ovvero un messaggio speciale, tra i processi. È disponibile un solo token e chiunque lo possieda può accedere alla risorsa condivisa. Dopo il completamento, il token viene trasferito al processo successivo. Se un processo che contiene il token non desidera accedere alla risorsa, la trasferisce direttamente a un altro processo.

Le soluzioni basate su token hanno diverse caratteristiche essenziali. In primo luogo, a seconda di come sono organizzati i processi, possono garantire che ogni processo abbia l'opportunità di accedere alla risorsa con relativa facilità. In altre parole, prevengono la starvation. In secondo luogo, i deadlock, in cui più processi aspettano indefinitamente che l'uno sia completato dall'altro, possono essere facilmente evitati, il che contribuisce alla loro semplicità. Quando il token viene perso (ad esempio, se il processo che lo contiene si arresta in modo anomalo), è necessario avviare una procedura distribuita complessa per garantire che venga generato un nuovo token, a condizione che sia anche l'unico token presente.

Molti algoritmi di mutua esclusione distribuiti utilizzano una strategia basata sulle autorizzazioni come alternativa. In questo caso, un processo che richiede l'accesso alla risorsa deve prima ottenere l'autorizzazione da altri processi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Algoritmo centralizzato di mutua esclusione

La simulazione di come viene eseguita in un sistema a processore singolo è un metodo semplice per ottenere la mutua esclusione in un sistema distribuito. Un processo è nominato coordinatore. Quando un processo desidera accedere a una risorsa condivisa, invia un messaggio di richiesta al coordinatore specificando la risorsa e richiedendo l'autorizzazione. Come illustrato nella Figura 1a, se nessun altro processo sta attualmente accedendo alla risorsa, il coordinatore restituisce una risposta che concede l'autorizzazione. Quando la risposta viene ricevuta, il richiedente può procedere.

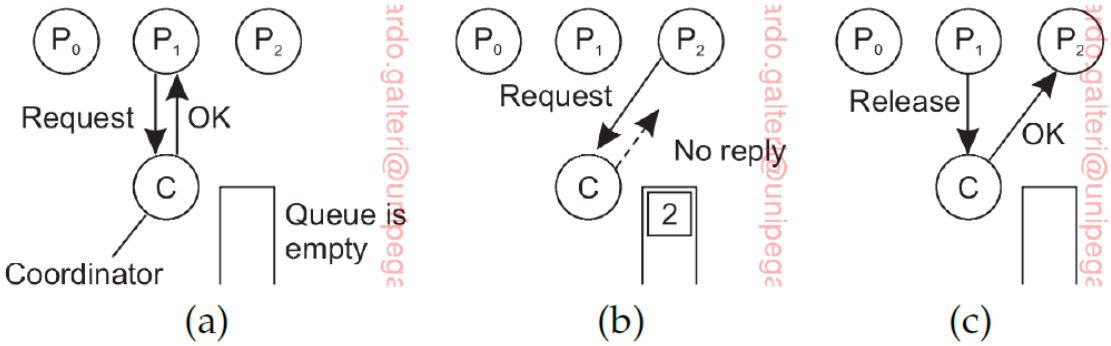


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Si supponga ora che un secondo processo, P_2 nella Figura 1b, richieda l'accesso alla risorsa. Il coordinatore è consapevole che un altro processo sta già utilizzando la risorsa, quindi non è possibile concedere l'autorizzazione. Il metodo preciso utilizzato per negare l'autorizzazione varia a seconda del sistema. Nella figura 1b, il coordinatore si astiene semplicemente dal rispondere, ostacolando così il processo P_2 in attesa, che è bloccato.

In alternativa, potrebbe inviare una risposta "autorizzazione negata". In entrambi i casi, la richiesta da P_2 viene temporaneamente accodata e vengono attesi ulteriori messaggi.

Come illustrato nella Figura 1c, quando il processo P_1 è terminato con la risorsa, invia un messaggio al coordinatore che rilascia il suo accesso esclusivo.

Il coordinatore prende il primo elemento dalla coda delle richieste e invia un messaggio di concessione al processo corrispondente. Se il processo è ancora bloccato (cioè, questo è il suo primo

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

messaggio), viene sbloccato e concesso l'accesso alla risorsa. Se è già stato inviato un messaggio esplicito che nega l'autorizzazione, il processo deve eseguire il polling del traffico in ingresso o bloccarlo in un secondo momento. Quando riceve l'autorizzazione, può procedere a prescindere.

È evidente che l'algoritmo garantisce l'esclusione reciproca: il coordinatore consente a un solo processo di accedere alla risorsa alla volta. È anche equo, in quanto le richieste sono concesse nell'ordine in cui sono state ricevute. Nessuna procedura aspetta mai eternamente (nessuna starvation). Inoltre, lo schema è semplice da implementare e richiede solo tre messaggi per utilizzo della risorsa (richiesta, concessione, rilascio). È una soluzione attraente per molte situazioni pratiche grazie alla sua semplicità.

Anche l'approccio centralizzato ha tuttavia dei difetti. Il coordinatore è un singolo punto di errore; pertanto, se fallisce, l'intero sistema potrebbe non funzionare. Se i processi normalmente si bloccano dopo aver inviato una richiesta, non possono distinguere tra un coordinatore morto e "autorizzazione negata" perché in entrambi i casi non viene restituito alcun messaggio. Inoltre, un singolo coordinatore può diventare un collo di bottiglia delle prestazioni in un sistema di grandi dimensioni. Tuttavia, i vantaggi della sua semplicità in genere superano qualsiasi potenziale svantaggio. Inoltre, le soluzioni distribuite non sono sempre superiori.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Algoritmo distribuito di mutua esclusione

Utilizzando i clock logici di Lamport e traendo ispirazione dalla soluzione originale di Lamport per l'esclusione reciproca distribuita, è stato sviluppato un algoritmo distribuito di mutua esclusione all'inizio degli anni '80. La soluzione richiede un ordinamento completo di tutti gli eventi di sistema. In altre parole, per qualsiasi coppia di eventi, come ad esempio i messaggi, deve essere chiaro quale si è verificato per primo.

L'algoritmo funziona come segue: quando un processo desidera accedere a una risorsa condivisa, costruisce un messaggio che include il nome della risorsa, il suo numero di processo e l'orario corrente (logico). Il messaggio viene quindi trasmesso a tutti i processi, includendo concettualmente sé stesso. Si presume che l'invio di messaggi sia affidabile, nel senso che nessun messaggio viene perso.

Quando un processo riceve un messaggio di richiesta da un altro processo, l'azione che esegue dipende dal proprio stato in relazione alla risorsa specificata nel messaggio. Si distinguono tre casi:

- Se il destinatario non accede alla risorsa e non desidera accedervi, invia un messaggio OK al mittente.
- Se il destinatario ha già accesso alla risorsa, non invia una risposta. Al contrario, mette in coda la richiesta.
- Se il destinatario desidera accedere alla risorsa ma non lo ha ancora fatto, il timestamp del messaggio in arrivo viene confrontato con il timestamp contenuto nel messaggio inviato a tutti. Vince quello con il punteggio più basso. Se il messaggio ricevuto ha un timestamp inferiore rispetto al messaggio in arrivo, il destinatario invia un messaggio OK. Se il messaggio del destinatario ha un timestamp inferiore, mette in coda la richiesta in arrivo e non la invia.

Dopo aver inviato le richieste di autorizzazione, un processo attende fino a quando tutti hanno concesso l'autorizzazione. Non appena vengono ricevute tutte le autorizzazioni, può procedere. Al termine, invia messaggi OK a tutti i processi nella coda e quindi li elimina tutti. Se non c'è conflitto, è evidentemente

efficace. Si supponga, tuttavia, che due processi tentino di accedere alla risorsa contemporaneamente, come illustrato nella Figura 2.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

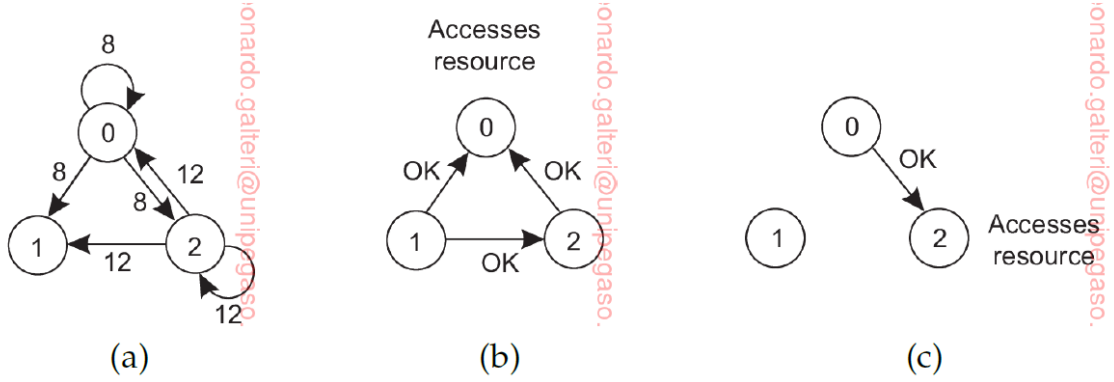


Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Il processo P_0 invia una richiesta con un timestamp di 8, mentre il processo P_2 invia una richiesta con un timestamp di 12. P_1 non è interessato alla risorsa, quindi risponde con un OK a entrambi i mittenti. Sia il processo P_0 che il processo P_2 osservano il conflitto e confrontano i timestamp. P_2 vede che ha perso, quindi invia OK a P_0 per concedere l'autorizzazione. Come illustrato nella Figura 2b, il processo P_0 ora mette in coda la richiesta da P_2 per l'elaborazione successiva e accede alla risorsa. La Figura 2c mostra che, una volta completata, rimuove la richiesta da P_2 dalla coda e invia un messaggio OK a P_2, consentendo a quest'ultima di procedere. In caso di conflitto, vince il timestamp con il valore più basso e tutti sono d'accordo sull'ordine dei timestamp.

Questo algoritmo garantisce l'esclusione reciproca senza deadlock o starvation. Se il numero totale di processi è N, un processo deve inviare e ricevere $2 \cdot (N-1)$ messaggi prima di entrare nella sua sezione critica: $N-1$ messaggi di richiesta a tutti gli altri processi, seguiti da $N-1$ messaggi OK, uno da ogni altro processo.

Sfortunatamente, questo algoritmo contiene N punti di fallimento. Nel caso in cui un processo si arresti in modo anomalo, non risponderà alle richieste. Questo silenzio sarà interpretato (erroneamente) come una negazione del permesso, impedendo successivi tentativi da parte di tutti i processi di accedere a una qualsiasi delle rispettive regioni critiche. L'algoritmo può essere modificato come descritto di seguito. Quando viene ricevuta una richiesta, il destinatario risponde sempre concedendo o negando l'autorizzazione. Quando una richiesta o una risposta viene persa, il mittente ritenta fino a quando non viene ricevuta una risposta o il mittente determina che la destinazione è fallita. Dopo che una richiesta viene rifiutata, il mittente deve bloccare fino a quando non viene ricevuto un successivo messaggio OK.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Un ulteriore problema con questo algoritmo è che deve essere utilizzata una primitiva di comunicazione multicast oppure ogni processo deve mantenere il proprio elenco di appartenenza al gruppo, inclusi i processi che si uniscono, escono e si arrestano in modo anomalo. Il metodo è più efficace con piccoli gruppi di processi le cui appartenenze non cambiano mai. Si noti che tutti i processi sono coinvolti in tutte le decisioni relative all'accesso alla risorsa condivisa, il che può comportare un onere per i processi in esecuzione su macchine con risorse limitate.

Diversi miglioramenti minori sono possibili per questo algoritmo. Ad esempio, ottenere il permesso da tutti è eccessivo. Tutto ciò che serve è un metodo per impedire a due processi di accedere contemporaneamente alla risorsa. L'algoritmo può essere modificato per concedere l'autorizzazione quando ha ottenuto l'autorizzazione dalla maggioranza semplice degli altri processi, anziché da tutti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Algoritmo token-ring

La figura 3 illustra un metodo completamente diverso per ottenere deterministicamente l'esclusione reciproca in un sistema distribuito. Nel software, una rete di overlay è costruita sotto forma di un anello logico in cui a ciascun processo viene assegnata una posizione. Tutto ciò che conta è che ogni processo conosca il suo immediato successore.

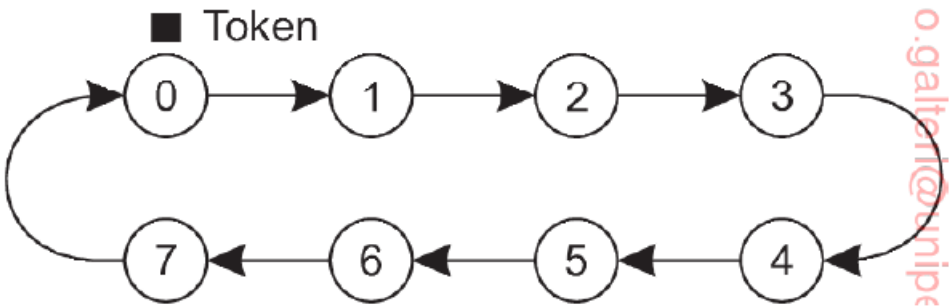


Figura 3 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Quando l'anello viene inizializzato, viene rilasciato un token al processo P_0. Il token si fa strada intorno all'anello. Supponendo che ci siano N processi, il token viene trasmesso tramite messaggi punto-a-punto dal processo P_k al processo $P_{(k+1) \bmod N}$.

Quando un processo acquisisce un token da un vicino, determina se richiede l'accesso a una risorsa condivisa. In tal caso, il processo continua, completa tutte le attività necessarie e rilascia le risorse. Dopo il completamento, il token viene passato lungo l'anello. Non è consentito utilizzare immediatamente lo stesso token per accedere nuovamente alla risorsa.

Se un processo riceve un token da un vicino ma non ha alcun interesse per la risorsa, passa semplicemente il token al processo successivo. Quando nessun processo richiede la risorsa, il token circola semplicemente attorno all'anello.

L'accuratezza di questo algoritmo è evidente. In un dato momento, solo un processo ha il token, quindi solo un processo può accedere alla risorsa. A causa della circolazione ben definita del token tra i processi, la starvation non può verificarsi. Una volta che un processo decide che ha bisogno di accedere a una risorsa, deve, nel peggiore dei casi, attendere fino a quando tutti gli altri processi hanno utilizzato la risorsa.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Questo algoritmo ha difetti intrinseci. Il token deve essere rigenerato in caso di smarrimento, ad esempio se il titolare si arresta in modo anomalo o se un messaggio contenente il token viene perso. In effetti, potrebbe essere difficile rilevare che il token è stato perso a causa della quantità illimitata di tempo tra le successive apparizioni del token sulla rete. Il fatto che il token non sia stato visto per un'ora non indica necessariamente che sia stato perso poiché potrebbe essere ancora in uso da qualcuno.

Quando un processo si arresta in modo anomalo, anche l'algoritmo incontra difficoltà, ma il recupero è relativamente semplice. Se richiediamo a un destinatario del token di confermare la ricezione, verrà rilevato un processo morto quando il suo vicino tenta e non riesce a dargli il token. A quel punto, il processo morto può essere rimosso dal gruppo e il titolare del token può inviare il token bypassando il processo morto, quindi consegnandolo al membro successivo in linea o, se necessario, al membro ancora successivo. Ovviamente, ciò richiede a tutti di preservare l'attuale configurazione ad anello.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Saxena P. and Rai J. A Survey of Permission-based Distributed Mutual Exclusion Algorithms. Computer Standards and Interfaces, 2003.
- Kshemkalyani A. and Singhal M. Distributed Computing, Principles, Algorithms, and Systems. Cambridge University Press, 2008.
- Ricart G. and Agrawala A. An Optimal Algorithm for Mutual Exclusion in Computer Networks. Communications of the ACM, 1981.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).